

Booking System Overbooking Problem

Race Condition and Database Transaction Solution

Student: Samandar Mirxosilov
Student ID: 230146
Course: Operating Systems
Date: December 29, 2025

1. Introduction

What is booking system?

Booking system is program that help people to reserve things like airplane tickets, hotel rooms, or concert tickets. The main problem happen when many people try to book same thing at same time.

Overbooking Problem

Overbooking is when system accept more reservations than available capacity. For example, airplane have 10 seats but system allow 15 people to book. This happen because of race condition - when multiple users book at same time without proper synchronization.

Centrum Air Case

Centrum Air is famous example of overbooking problem. Their booking system had big bug that allow multiple agents to book same seats together.

What happened:

- Many booking agents use system at same time
- All agents see "seats available" for same flight
- All agents complete booking in same moment
- Result: More bookings than seats!

This cause big problems:

- Angry passengers cannot board plane
- Company must pay compensation (expensive!)
- Legal problems and lawsuits
- Company reputation damaged
- Customers lose trust

This real case show why we need proper synchronization in concurrent systems.

2. Critical Section

What is Critical Section?

Critical section is part of code that access shared resources. Shared resource mean data that multiple threads use together. In booking system, the shared resource is "available_seats" number in database.

Why it matters?

When multiple threads run critical section at same time without protection:

- Data can be corrupted
- System state become inconsistent
- Race condition happens
- Results are unpredictable

Critical Section in My Project

...

CRITICAL SECTION:

- Read available_seats from database
- Check if available_seats > 0

- Decrease available_seats by 1
- Write new value to database
- Insert booking record

...

****The Problem:**** If two threads enter this section together, both will read same value (for example "10 seats"), both will think seats available, both will book, but only one seat should be taken!

Requirements for Critical Section Protection

To protect critical section properly we need:

1. ****Mutual Exclusion**** - Only one thread can be in critical section at time
2. ****Progress**** - If nobody in critical section, waiting threads should enter
3. ****Bounded Waiting**** - Thread should not wait forever
4. ****No Hardware Assumptions**** - Should work on any computer

3. Race Condition Problem

What is Race Condition?

Race condition happen when two or more threads access shared data at same time, and at least one thread modify the data. The final result depend on timing and execution order, so it's unpredictable.

Race Condition in My Booking System

****Example:**** 10 seats available, 3 customers try to book together

...

Time	Thread 1	Thread 2	Thread 3	Database
t1	Read: 10 seats			seats=10
t2		Read: 10 seats		seats=10
t3			Read: 10 seats	seats=10
t4	Check: 10>0 Yes!			
t5		Check: 10>0 Yes!		
t6			Check: 10>0 Yes!	
t7	Calculate: 10-1=9			
t8		Calculate: 10-1=9		
t9			Calculate: 10-1=9	
t10	Write: 9 seats			seats=9
t11		Write: 9 seats		seats=9
t12			Write: 9 seats	seats=9
t13	Insert booking			booking1
t14		Insert booking		booking2
t15			Insert booking	booking3

RESULT: 3 bookings made, but database show 9 seats!
Should be 7 seats! OVERBOOKING!

...

Why This Happens?

Race condition occur because of ****Read-Modify-Write**** pattern:

1. ****Read**** - All threads read same value together
2. ****Modify**** - Each thread calculate based on old value
3. ****Write**** - All threads write their results, overwriting each other

****Main Problem:**** Time gap between READ and WRITE allow other threads to interfere.

Read/Write Conflict

Read/Write conflict happen when one thread reading data and another thread writing same data at same time with no synchronization.

****Three types of conflicts:****

1. ****Read-Read**** - Safe (both just reading)
2. ****Read-Write**** - Dangerous (reader get wrong data)
3. ****Write-Write**** - Critical (data corruption!)

My booking system have Read-Write and Write-Write conflicts together.

4. Producer-Consumer Analogy

The Classic Problem

Producer-Consumer is famous synchronization problem:

- Producers create data and put in shared buffer
- Consumers take data from shared buffer
- Both access same buffer at same time

My Booking System as Producer-Consumer

****In booking system:****

- ****Producers**** = Customers making booking requests
- ****Consumer**** = Database system processing requests
- ****Shared Buffer**** = Available seats counter

The Problem Without Synchronization

```

Customer 1: "I want seat!" (see 10 available)  
Customer 2: "I want seat!" (see 10 available)  
Customer 3: "I want seat!" (see 10 available)  
...all book at same time...  
Database: "Processing..." (chaos!)  
Result: Buffer overflow = overbooking  
```

With Synchronization

```

Customer 1: "I want seat!" â†’ LOCK â†’ Process â†’ UNLOCK (9 left)  
Customer 2: "I want seat!" â†’ WAIT...LOCK â†’ Process â†’ UNLOCK (8 left)  
Customer 3: "I want seat!" â†’ WAIT...LOCK â†’ Process â†’ UNLOCK (7 left)  
Result: Orderly processing, no overbooking  
```

Just like producers and consumers need mutex to access shared buffer safely, my booking system need synchronization to ensure seat count stay accurate and no overbooking occur.

5. Solution: Database Transaction

What is Transaction?

Transaction is group of database operations that treated as single unit.

Transaction follow ACID properties:

- ****Atomicity**** - All operations complete or none (all-or-nothing)
- ****Consistency**** - Database always in valid state
- ****Isolation**** - Transactions don't interfere with each other
- ****Durability**** - Completed transactions saved forever

Transaction Isolation

I use EXCLUSIVE mode in SQLite. This provide strongest isolation - it's like SERIALIZABLE level. This mean transactions execute as if they run one after another, not together.

How EXCLUSIVE Lock Works

...

Thread 1 start transaction

```
|
|- BEGIN EXCLUSIVE â†’ Lock database
|                               Database is LOCKED now!
|
|- Read seats: 10
|- Check availability
|- Update seats: 9
|- Insert booking
|
â””- COMMIT â†’ Release lock
|                               Database UNLOCKED now
```

Thread 2 must wait...

```
|
â””- Can only start after Thread 1 finish
...
```

Safe Implementation

****Main differences from unsafe version:****

****Unsafe (no protection):****

```
```python
conn = sqlite3.connect('booking.db')
read seats
process booking
write seats
commit
```
```

****Safe (with transaction):****

```
```python
conn = sqlite3.connect('booking.db', isolation_level='EXCLUSIVE')
BEGIN EXCLUSIVE # Lock database!
read seats # Protected
process booking # Protected
write seats # Protected
COMMIT # Unlock database
```
```

Why This Works?

- ****Mutual Exclusion:**** Only one transaction hold EXCLUSIVE lock at time
- ****Atomicity:**** If something fail, entire transaction rollback
- ****Consistency:**** Database state always valid (never negative seats)
- ****Isolation:**** Other transactions wait until current one complete

6. My Code Implementation

Database Structure

I created two tables in SQLite:

```
**Table: flights**
- flight_number: "CA100"
- total_seats: 10
- available_seats: 10
```

```
**Table: bookings**
- customer_name: "Customer_1"
- flight_number: "CA100"
- booking_time: timestamp
```

Unsafe Version (WITH race condition)

```
```python
def book_flight_unsafe(flight_number, customer_name):
 conn = sqlite3.connect('booking.db')
 cursor = conn.cursor()

 # Read seats
 cursor.execute('SELECT available_seats FROM flights WHERE flight_number
= ?')
 available_seats = result[0]

 # PROBLEM HERE! Other threads also reading!
 time.sleep(random.uniform(0.01, 0.05))

 # Book if available
 if available_seats > 0:
 new_seats = available_seats - 1
 cursor.execute('UPDATE flights SET available_seats = ?', (new_seats,))
 cursor.execute('INSERT INTO bookings VALUES (?)', (customer_name,))

 conn.commit()
```
```

```
**Why it fails:**
- No locking
- Time gap between read and write
- Multiple threads read same value
- Last write wins (data lost!)
```

Safe Version (WITHOUT race condition)

```
```python
def book_flight_safe(flight_number, customer_name):
 conn = sqlite3.connect('booking.db', isolation_level='EXCLUSIVE')
 cursor = conn.cursor()

 try:
 cursor.execute('BEGIN EXCLUSIVE') # LOCK!

 # Read (protected now)
 cursor.execute('SELECT available_seats FROM flights WHERE flight_number
= ?')
 available_seats = result[0]

 time.sleep(random.uniform(0.01, 0.05)) # Safe now!

 # Book if available
 if available_seats > 0:
 new_seats = available_seats - 1
 cursor.execute('UPDATE flights SET available_seats = ?',
(new_seats,))
 cursor.execute('INSERT INTO bookings VALUES (?)', (customer_name,))
 except:
```

```

 conn.commit() # UNLOCK!
 except:
 conn.rollback() # UNLOCK on error
...

```

**\*\*Why it works:\*\***

- EXCLUSIVE lock prevent concurrent access
- Entire operation is atomic
- Rollback on error
- Other threads must wait
- Data integrity guaranteed!

### Testing with Multithreading

I test both implementations with multiple threads:

```

```python
def test_booking(num_customers=15):
    threads = []

    # Create threads
    for i in range(15):
        thread = threading.Thread(target=book_flight, args=(customer_i))
        thread.start() # All start TOGETHER!

    # Wait for all to finish
    for thread in threads:
        thread.join()
...

```

This create 15 threads all trying to book simultaneously - maximum chance for race condition!

7. Test Results

Test Setup

- Flight: CA100
- Total seats: 10
- Booking attempts: 15 (concurrent)
- Method: Python threading

Unsafe Booking Results

****Test 1:****

...

```

Booking attempts: 15
Successful: 13
Total in DB: 13
Remaining seats: -3
â§ i,  OVERBOOKING: 3 extra!
...

```

****Test 2:****

...

```

Booking attempts: 15
Successful: 12
Total in DB: 12
Remaining seats: -2
â§ i,  OVERBOOKING: 2 extra!
...

```

```

**Analysis:**
- Results different each time (non-deterministic)
- Always produce overbooking
- Negative seats (impossible!)
- Clear race condition

```

Safe Booking Results

```

**Test 1:**
```

```

```

Booking attempts: 15
Successful: 10
Failed: 5
Total in DB: 10
Remaining seats: 0
â€¦ CORRECT!
```

```

```

**Test 2:**
```

```

```

Booking attempts: 15
Successful: 10
Failed: 5
Total in DB: 10
Remaining seats: 0
â€¦ CORRECT!
```

```

```

**Analysis:**
- Results same every time (deterministic)
- Exactly 10 bookings
- 5 customers correctly rejected
- No race condition

```

Comparison

Metric	Unsafe	Safe
Correctness	â€¦ Fail	â€¦ Pass
Consistency	â€¦ Varies	â€¦ Always same
Overbooking	Yes (2-5)	No
Speed	~0.15s	~0.20s

Safe version is bit slower because of locking, but correctness more important than small performance difference!

8. Real World Impact

Business Problems from Overbooking

```

**Money costs:**
- Pay compensation to customers ($400-1350 per person for airlines)
- Hotel for stranded passengers
- Food and transport
- Administrative work

```

```

**Reputation damage:**
- Bad social media posts
- Customer trust lost
- Competitors advantage
- Government attention

```

****Legal problems:****

- Lawsuits from customers
- Government fines
- Contract violations

Why Technical Solutions Important

Centrum Air case show that software bugs cause real problems:

- People stranded, miss important events
- Company lose millions of dollars
- Brand destroyed on social media

My solution prevent this by:

1. Ensure data integrity with transactions
2. Prevent race conditions with locking
3. Maintain consistency with ACID
4. Provide atomic operations

Other Systems with Same Problem

Same problem appear in:

- Banking (multiple withdrawals from one account)
- E-commerce (last item in stock)
- Healthcare (double-booking appointments)
- Concert tickets
- Warehouse inventory

All need similar synchronization solutions.

9. Conclusion

What I Learned

This project show me:

1. Race conditions are real problem
2. Concurrent access need synchronization
3. Testing with multiple threads reveal bugs
4. Proper locking essential for data integrity
5. Performance secondary to correctness

Project Results

I successfully:

- Created working booking system
- Demonstrated race condition (unsafe version)
- Implemented transaction solution (safe version)
- Tested with multithreading
- Proved solution works

Understanding I Gained

Now I understand:

- What critical sections are
- How race conditions occur
- Why synchronization important
- How database transactions prevent corruption
- Real-world impact of concurrency bugs

Final Thought

Centrum Air overbooking not just theory - it's real failure that cost millions and hurt real people. Understanding race conditions and using proper

synchronization not optional - it's fundamental requirement for reliable software.

My project prove that with correct understanding and proper use of database transactions, we can build systems that work correctly even under heavy concurrent load.

****Most important lesson: Correctness always before performance. Slow system better than broken system.****

References

1. Operating System Concepts textbook
2. Python threading documentation
3. SQLite transaction documentation
4. Centrum Air case studies
5. ACID properties articles
